Control Barrier Function Safety Filters for Mobile Robots: An Exact Discrete-Time Guarantee and the Role of the Barrier Rate

Mahim Trivedi
Python Robotics Projects, github.com/robofan45/cbf-safety-filter
November 2026

***Abstract*—A safety filter separates the two jobs of a robot controller: a nominal controller pursues the task, and the filter changes its command as little as possible so that the robot stays in a safe set. Control barrier functions (CBFs) make this precise: a barrier that is positive on the safe set may decrease, but never faster than in proportion to its own value, and the safe command closest to the nominal one is the solution of a small quadratic program. This report presents a compact CBF safety filter for a planar robot among circular obstacles, with an exact closed-form solver for the two-dimensional quadratic program, and shows that for this barrier and robot model the sampled implementation keeps every straight segment of the path inside the safe set, not merely the samples. In the demonstration, a go-to-goal controller that would drive through all three obstacles reaches its goal in 19.2 s without touching any of them; the filter changes 269 of 383 commands and leaves every one of the 114 commands that were already safe exactly as they were. A sweep of the barrier rate shows the central trade-off: as the rate grows from 0.25 to 6 per second, the time to the goal falls from 25.8 s to 18.3 s, while the closest approach to an obstacle falls from 35 cm to 0.1 mm. The exact solver agrees with a general-purpose optimiser to 5×10^{-8} on 1000 random problems and is three times faster.**

***Index Terms*—*control barrier functions, safety-critical control, quadratic programming, set invariance, collision avoidance, mobile robots*

I. Introduction

A robot that works near people or obstacles must be safe whatever its task-level controller asks it to do. A common way to achieve this is to separate the two concerns. A nominal controller is designed for performance, without regard to safety, and a safety filter sits between it and the actuators, passing its commands through when they are safe and changing them as little as possible when they are not [1]. The filter can be verified on its own, and the nominal controller can be redesigned, tuned or learned without touching it.

Control barrier functions (CBFs) give such filters a precise form [1], [2]. The safe set is described as the set where a function, the barrier, is non-negative. A command is admissible if it lets the barrier decrease no faster than in proportion to its current value, so that the barrier may approach zero but never cross it. For systems whose dynamics are affine in the input this is a linear constraint on the command, and the admissible command closest to the nominal one is the solution of a quadratic program (QP) that can be solved at every control step.

This report describes a compact CBF safety filter for a robot that moves in the plane among circular obstacles, driven by a go-to-goal controller that knows nothing about them. Its contributions are:

- an implementation of the filter with an exact, closed-form solver for the two-dimensional QP, which needs no optimisation library;
- a short proof that, for this barrier and robot model, the sampled implementation keeps every straight segment of the path inside the safe set, a stronger property than the continuous-time guarantee usually quoted;
- an evaluation of the trade-off controlled by the barrier rate, between progress and the margin kept from the obstacles;
- tests that check safety, minimal intervention and the solver against a general-purpose optimiser.

II. Related Work

The idea that a set is invariant if the dynamics never point out of it on its boundary goes back to Nagumo [3]; Blanchini surveys its use in control [4]. Prajna and Jadbabaie introduced barrier certificates to verify the safety of a given system [5], and Wieland and Allgöwer turned the idea into a design tool, a barrier condition that a controller must satisfy, which they called a control barrier function [6]. Ames et al. relaxed the condition so that the barrier may decrease towards the boundary at a controlled rate, and combined it with control Lyapunov functions in a QP that trades off performance against safety [2]; their tutorial [1] collects the theory and applications, including the minimally invasive safety filter used here.

CBFs have been applied to collision avoidance among multiple robots [7], to bipedal walking [8] and to adaptive cruise control [2]. When the input does not act directly on the barrier, as for a position barrier on a robot controlled through its acceleration, the barrier has a relative degree higher than one, and exponential [9] or high-order [10] CBFs are needed. Agrawal and Sreenath formulated CBFs directly for discrete-time systems [8]. The QP-based filter can also misbehave: Reis, Aguiar and Tabuada showed that it can introduce undesirable equilibria in which the robot stops in front of an obstacle [11], an effect visible in a milder form in Section VI. Robustness to model and measurement errors is addressed by input-to-state safety [12].

An older alternative is the artificial potential field, which adds a repulsive term to the nominal controller [13]. It needs no optimisation but changes the command everywhere near an obstacle, gives no guarantee that the obstacle is never entered, and suffers from local minima. A CBF filter, by contrast, leaves safe commands untouched and guarantees safety by construction.

III. Problem Formulation

A. Robot and obstacles

The robot is a disc of radius r_0 whose centre $p \in \mathbb{R}^2$ moves with the commanded velocity u, a single integrator. Commands are held constant over the control period Δt, so

$$p_{t+1} = p_t + u_t \Delta t \tag{1}$$

The obstacles are discs with centres c_i and radii r_i, $i = 1, \dots, M$. Adding the robot's radius to each obstacle reduces the problem to keeping the point p out of discs of radius $R_i = r_i + r_0$. Each of them gives a barrier

$$h_i(p) = \|p - c_i\|^2 - R_i^2 \tag{2}$$

which is positive while the robot is clear of the obstacle, zero when it touches it and negative when it overlaps it. The safe set is the set of positions where every h_i is non-negative.

B. Nominal controller

The nominal controller drives straight at the goal g with a proportional law, slowing down near the goal and saturated at the speed $v_{\max}$:

$$u_{\text{nom}} = K(g - p) \min\left(1, \frac{v_{\max}}{\|K(g - p)\|}\right) \tag{3}$$

It knows nothing about the obstacles. In the scenario studied here, the start is at the origin, the goal at (10 m, 0.8 m), and three obstacles lie across the straight line between them, each slightly off it so that it has a preferred side (Table I). Driven by the nominal controller alone, the robot would pass through all three: the smallest barrier value along its path is -1.40 m².

IV. METHOD

A. The control barrier function condition

The rate of change of a barrier along the motion is $\dot{h}_i = \nabla h_i(p)^\top u$. The CBF condition requires, for every obstacle,

$$\dot{h}_i = 2(p - c_i)^\top u \geq -\alpha\, h_i(p) \tag{4}$$

with a barrier rate $\alpha > 0$. Far from an obstacle, h_i is large and the condition allows the robot to approach it quickly; close to it, the allowed rate of approach shrinks in proportion to the remaining margin. In continuous time, a comparison argument gives

$$h_i\big(p(t)\big) \geq h_i\big(p(0)\big)e^{-\alpha t} \tag{5}$$

so a barrier that starts positive stays positive [1], [2]. The rate α sets how late and how hard the robot brakes: a small α makes it slow down far from the obstacle, a large one lets it come close before the condition bites.

B. The safety filter

The filter chooses the command closest to the nominal one among those that satisfy the condition for every obstacle:

$$u^* = \arg\min_u \|u - u_{\text{nom}}\|^2 \ \text{subject to}\ 2(p - c_i)^\top u \geq -\alpha\, h_i(p), \qquad i = 1, \ldots, M \tag{6}$$

The constraints are linear in u, so this is a convex QP with a strictly convex objective and therefore a unique solution whenever it is feasible. For a single integrator, feasibility is never in doubt: while the robot is clear of every obstacle, all right-hand sides are negative and the command $u = 0$, standing still, satisfies every constraint. The filter is also minimally invasive: if the nominal command satisfies every constraint, it is the solution and passes unchanged.

C. An exact solver for two-dimensional commands

General QP solvers iterate to a tolerance. With a two-dimensional command the problem can be solved exactly by enumeration. At the solution, the active constraints with linearly independent normals number at most two, so the solution is one of three kinds of candidate: the nominal command itself, if no constraint is active; its projection onto one constraint line, if one is,

$$u_i = u_{\text{nom}} + \frac{b_i - a_i^\top u_{\text{nom}}}{\|a_i\|^2}\, a_i \tag{7}$$

where $a_i = 2(p - c_i)$ and $b_i = -\alpha\, h_i(p)$; or the intersection of two constraint lines, if two are. The solver forms all of these candidates, discards those that violate a constraint by more than 10^{-9}, and returns the feasible candidate closest to the nominal command. Because the problem is convex, the closest feasible candidate is the global minimum. With M obstacles there are $1 + M + \frac{M(M-1)}{2}$ candidates, a handful for the three obstacles here.

D. A guarantee for the sampled system

The continuous-time argument of Section IV-A does not by itself cover the implementation, in which each command is held for a whole control period. For this barrier and this robot model, however, an exact discrete-time guarantee follows from convexity. Each h_i is a convex function of p, so it lies above its tangent plane. For a point a fraction $s \in [0,1]$ of the way along the step,

$$h_i(p + su\Delta t) \geq h_i(p) + s\, \nabla h_i(p)^\top u\, \Delta t \geq (1 - s\alpha\Delta t) h_i(p) \tag{8}$$

where the second inequality is the CBF condition. As long as $\alpha\Delta t < 1$, the right-hand side is positive for every s, so the whole straight segment travelled during the step lies in the safe set, not only its end point. Applied step after step, the bound shows that each barrier decays at most geometrically,

$$h_i(p_t) \geq (1 - \alpha\Delta t)^t h_i(p_0) > 0 \tag{9}$$

the discrete counterpart of the exponential bound above. With the values used here, $\alpha\Delta t = 0.075$. The argument relies on two properties, a barrier that is convex in the position and an input that moves the position directly, and does not carry over to robots with inertia or nonholonomic constraints. Algorithm 1 summarises one control step.

Algorithm 1. One step of the CBF safety filter

1: Compute the nominal command u_{nom} from the current position and the goal.
2: For each obstacle, compute the constraint row $a_i = 2(p - c_i)$ and bound $b_i = -\alpha\, h_i(p)$.
3: Form the candidates: u_{nom}, its projection onto each line $a_i^{\top} u = b_i$, and the intersection of each pair of lines.
4: Keep the candidates that satisfy every constraint, and choose the one closest to u_{nom}.
5: Apply the chosen command for one control period: $p \leftarrow p + u^{*}\Delta t$.

V. Implementation

The filter is a single Python module of about 120 lines built on NumPy [14]; the obstacles are rows of an array, so the constraint rows and bounds of all obstacles are computed at once. It grew out of the PythonRobotics collection [15] and follows its conventions: running the module animates the robot together with its nominal and filtered commands and plots the clearance to each obstacle. Table I lists the parameters.

TABLE I
Parameters of the scenario and the filter

Parameter	Symbol	Value
Control period	Δt	0.05 s
Barrier rate (demonstration)	α	1.5 s^{-1}
Nominal gain, speed limit	K, $v_{\max}$	1.0 s^{-1}, 1.0 m/s
Robot radius	r_0	0.3 m
Start, goal	p_0, g	(0, 0) m, (10, 0.8) m
Goal tolerance		0.05 m
Obstacle 1: centre, radius	c_1, r_1	(3.0, −0.3) m, 1.0 m
Obstacle 2: centre, radius	c_2, r_2	(6.0, 1.0) m, 0.8 m
Obstacle 3: centre, radius	c_3, r_3	(8.5, 0.3) m, 0.6 m

Six automated tests check the properties claimed above rather than only running the code. They assert that the robot reaches the goal; that every barrier stays positive at every step; that without the filter the robot would collide, so that the scenario actually tests the filter; that every command the nominal controller produces that already satisfies the constraints passes unchanged, while every other one is changed; that the constraint rows equal the barrier gradients computed by finite differences; and that the exact QP solver agrees with SciPy's sequential quadratic programming solver SLSQP [16], [17] on random problems.

VI. Experiments

A. Demonstration

Fig. 1 shows the path of the robot with the filter and the straight path the nominal controller would take on its own. With the filter, the robot reaches the goal after 19.15 s without entering any of the keep-out circles; the closest it comes is 4.3 mm, and the smallest barrier value is 0.0095 m², positive as the analysis requires. The filter changes 269 of the 383 commands. The remaining 114 were already safe, and every one of them is applied exactly as the nominal controller produced it.

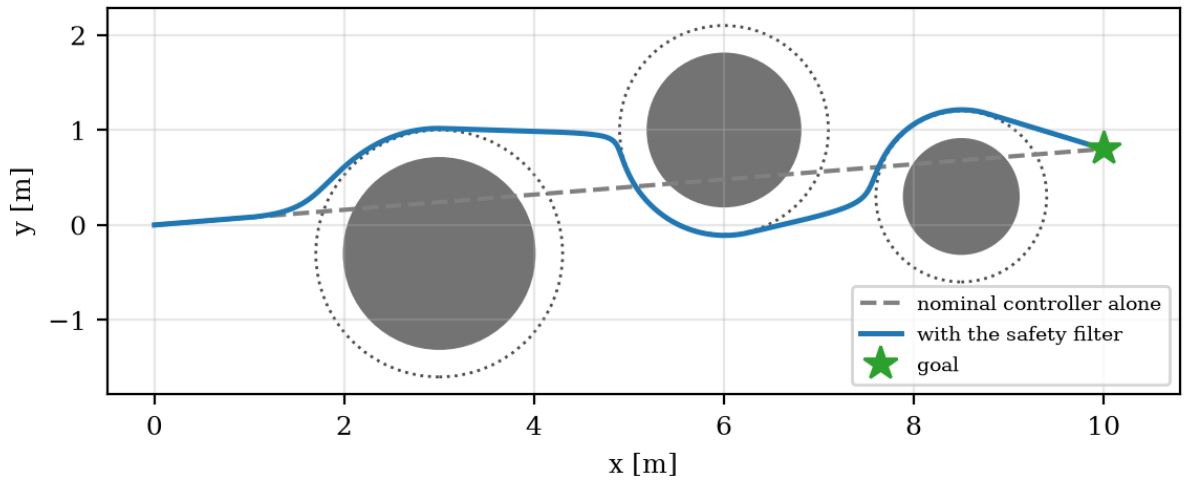


Fig. 1. Path of the robot with the safety filter (blue) and the path of the nominal controller alone (dashed), which runs through all three obstacles. Grey discs: obstacles; dotted circles: keep-out circles, enlarged by the robot's radius.

Fig. 2 shows the clearance to each obstacle and the size of the change the filter makes to the nominal command. The filter acts in three intervals, one per obstacle, and is inactive in between; the change reaches 0.95 m/s, almost the full nominal speed. In each interval the robot first approaches the keep-out circle, braking as the condition tightens, and then slides along it until the nominal command points away from the obstacle again. Around the second obstacle this takes about four seconds: the robot meets the keep-out circle at a point where the nominal command points almost exactly at the obstacle's centre, so after the filter has removed the component of the command towards the obstacle, little is left to move the robot along the circle. This is a mild form of the undesirable equilibrium described in [11], and it is where most of the time lost to safety is spent.

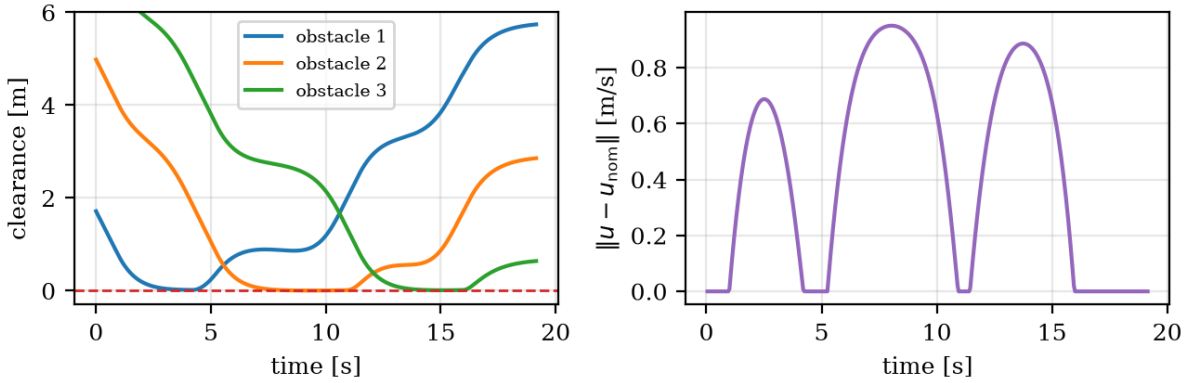


Fig. 2. Left: clearance between the robot and each keep-out circle; the dashed line marks contact. Right: size of the change the filter makes to the nominal command.

B. The barrier rate

Table II and Fig. 3 show the effect of the barrier rate α. A small rate makes the robot react to obstacles early and keep well away from them. At $\alpha = 0.25$, the filter changes 91% of the commands, the robot passes the second obstacle on the other side, and it reaches the goal after 25.8 s with a closest approach of 35 cm. As the rate grows, the robot reacts later and comes closer: at $\alpha = 6$ it reaches the goal after 18.25 s, changes only 57% of the commands, and passes the obstacles at 0.1 mm. Every run is safe; what changes is how much of the available room the robot uses.

TABLE II
EFFECT OF THE BARRIER RATE ON PROGRESS AND CLEARANCE

Barrier rate α [1/s]	Time to goal	Closest approach	Smallest barrier	Commands changed	Mean change
0.25	25.80 s	347 mm	0.885 m²	472 of 516	0.59 m/s
0.5	22.30 s	40 mm	0.091 m²	385 of 446	0.51 m/s
1.0	20.50 s	11 mm	0.025 m²	326 of 410	0.47 m/s
1.5 (demo)	19.15 s	4.3 mm	0.0095 m²	269 of 383	0.43 m/s
3.0	18.45 s	0.62 mm	0.0014 m²	225 of 369	0.41 m/s
6.0	18.25 s	0.10 mm	0.0002 m²	209 of 365	0.41 m/s

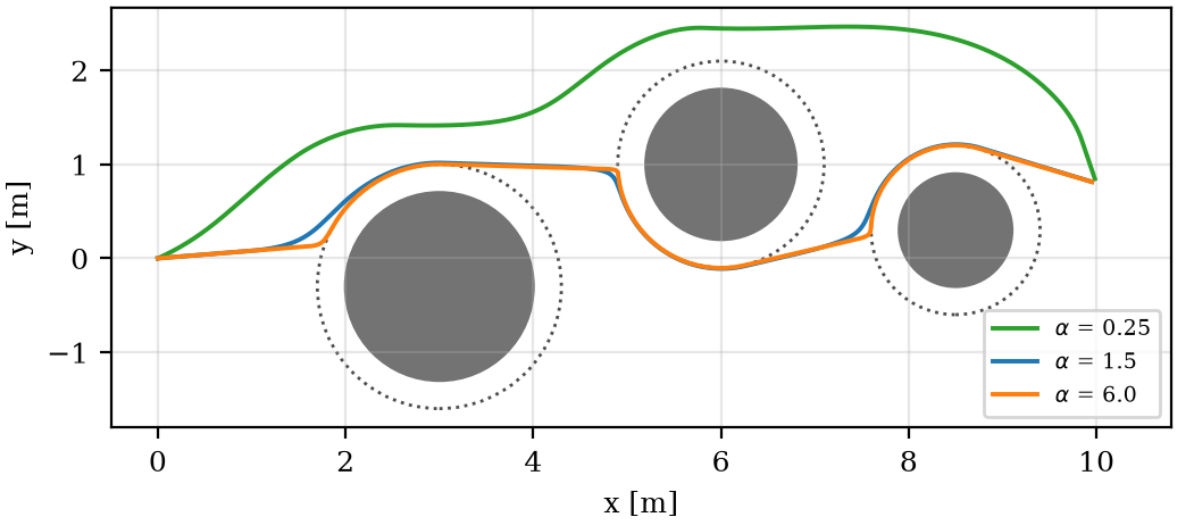


Fig. 3. Paths for three barrier rates. With the smallest rate (green) the robot reacts so early that it passes the second obstacle on the other side.

The closest approach shrinks by more than three orders of magnitude across the sweep, and the reason is the sliding motion described above. While the robot slides along a keep-out circle, the constraint of that obstacle is active and its barrier decays by the factor $1 - \alpha\Delta t$ at every step, so after a sliding phase of duration T the barrier has fallen to about $e^{-\alpha T}$ of its value at the start of the phase. The margin therefore shrinks exponentially in the product of the rate and the sliding time, but it never reaches zero. The practical lesson is that a CBF filter uses all the room it is given: any margin for sensing or actuation errors must be built into the keep-out radius, not expected from the filter.

C. The exact solver

To verify the solver, it was compared with SciPy's SLSQP [16], [17], run with a tolerance of 10^{-14}, on 1000 random problems with one to four constraints and a feasible origin. The largest difference between the two solutions is 5.1×10^{-8}, which is the accuracy of the iterative solver; the exact solver has no

tolerance to set. It also takes 59 µs per problem against 178 µs for SLSQP, both called from Python. Table III summarises the measurements.

TABLE III
SUMMARY OF THE MEASUREMENTS

Measure	Value
Smallest barrier on the nominal path (no filter)	−1.40 m²
Time to goal with the filter (α = 1.5)	19.15 s
Closest approach, smallest barrier	4.3 mm, 0.0095 m²
Commands changed by the filter	269 of 383
Safe nominal commands changed	0 of 114
Largest difference from SLSQP, 1000 random problems	5.1×10^{-8}
Time per problem: exact solver, SLSQP	59 µs, 178 µs

VII. Discussion

The guarantee of Section IV-D rests on a model in which the robot can change its velocity instantly, so that standing still is always possible and always safe. Real robots have inertia: if the command is an acceleration, a position barrier has relative degree two, braking takes distance, and feasibility of the QP is no longer automatic. Exponential and high-order CBFs [9], [10] address this by constraining a combination of the barrier and its derivatives, and the discrete-time formulation of Agrawal and Sreenath [8] avoids relying on the continuous-time argument altogether. Extending the present filter to a unicycle or a car-like model is the natural next step.

The filter guarantees safety, not progress. Near the second obstacle it slowed the robot to a crawl because the nominal command pointed almost exactly at the obstacle, and with perfect symmetry the robot would have stopped for good [11]. The obstacles in the demonstration are placed slightly off the straight line to avoid this, which is a property of the scenario rather than of the method. In practice the remedy lies outside the filter: a planner that supplies a nominal path around the obstacles, or a nominal controller with a tangential term. The same myopia explains why the robot passes the second obstacle on different sides for different barrier rates: the filter decides nothing about the route, which is decided by where its constraints first become active.

Finally, the measured margins are those of a perfect model with exact positions. The filter places the robot as close to the boundary as the barrier rate allows, so measurement noise and model errors would turn a margin of a tenth of a millimetre into a collision. Input-to-state safety [12] quantifies how much a disturbance can erode the guarantee, and the simplest robust design is the one used here: enlarge the keep-out circles by the expected error.

VIII. Conclusion

This report presented a control barrier function safety filter for a planar robot among circular obstacles, with an exact solver for its quadratic program and a proof that, for this model, the sampled filter keeps the whole path, not only its samples, inside the safe set. In the demonstration, a nominal controller that would cross all three obstacles reaches its goal safely, and only the commands that were unsafe are changed. The barrier rate trades progress against margin over a wide range, from 25.8 s and 35 cm to 18.25 s and 0.1 mm, and in every case the filter uses all of the margin it is allowed. The code, the tests and the script that regenerates every figure and number in this report are available at github.com/robofan45/cbf-safety-filter.

ACKNOWLEDGMENT

The software and this report were developed with the assistance of an AI coding assistant (Claude, by Anthropic). All results were produced by the code in the repository and can be regenerated with scripts/make_figures.py.

REFERENCES

[1] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *Proc. Eur. Control Conf. (ECC)*, 2019, pp. 3420–3431.

[2] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Trans. Autom. Control*, vol. 62, no. 8, pp. 3861–3876, 2017.

[3] M. Nagumo, "Über die Lage der Integralkurven gewöhnlicher Differentialgleichungen," *Proc. Phys.-Math. Soc. Jpn.*, vol. 24, pp. 551–559, 1942.

[4] F. Blanchini, "Set invariance in control," *Automatica*, vol. 35, no. 11, pp. 1747–1767, 1999.

[5] S. Prajna and A. Jadbabaie, "Safety verification of hybrid systems using barrier certificates," in *Hybrid Systems: Computation and Control (HSCC)*, Lecture Notes in Computer Science, vol. 2993. Berlin, Germany: Springer, 2004, pp. 477–492.

[6] P. Wieland and F. Allgöwer, "Constructive safety using control barrier functions," *IFAC Proc. Volumes*, vol. 40, no. 12, pp. 462–467, 2007.

[7] L. Wang, A. D. Ames, and M. Egerstedt, "Safety barrier certificates for collisions-free multirobot systems," *IEEE Trans. Robot.*, vol. 33, no. 3, pp. 661–674, 2017.

[8] A. Agrawal and K. Sreenath, "Discrete control barrier functions for safety-critical control of discrete systems with application to bipedal robot navigation," in *Proc. Robot.: Sci. Syst. (RSS)*, 2017.

[9] Q. Nguyen and K. Sreenath, "Exponential control barrier functions for enforcing high relative-degree safety-critical constraints," in *Proc. Amer. Control Conf. (ACC)*, 2016, pp. 322–328.

[10] W. Xiao and C. Belta, "Control barrier functions for systems with high relative degree," in *Proc. IEEE Conf. Decis. Control (CDC)*, 2019, pp. 474–479.

[11] M. F. Reis, A. P. Aguiar, and P. Tabuada, "Control barrier function-based quadratic programs introduce undesirable asymptotically stable equilibria," *IEEE Control Syst. Lett.*, vol. 5, no. 2, pp. 731–736, 2021.

[12] S. Kolathaya and A. D. Ames, "Input-to-state safety with control barrier functions," *IEEE Control Syst. Lett.*, vol. 3, no. 1, pp. 108–113, 2019.

[13] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *Int. J. Robot. Res.*, vol. 5, no. 1, pp. 90–98, 1986.

[14] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[15] A. Sakai, D. Ingram, J. Dinius, K. Chawla, A. Raffin, and A. Paques, "PythonRobotics: A Python code collection of robotics algorithms," arXiv:1808.10703, 2018.

[16] D. Kraft, "A software package for sequential quadratic programming," DFVLR, Cologne, Germany, Tech. Rep. DFVLR-FB 88-28, 1988.

[17] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nat. Methods*, vol. 17, pp. 261–272, 2020.